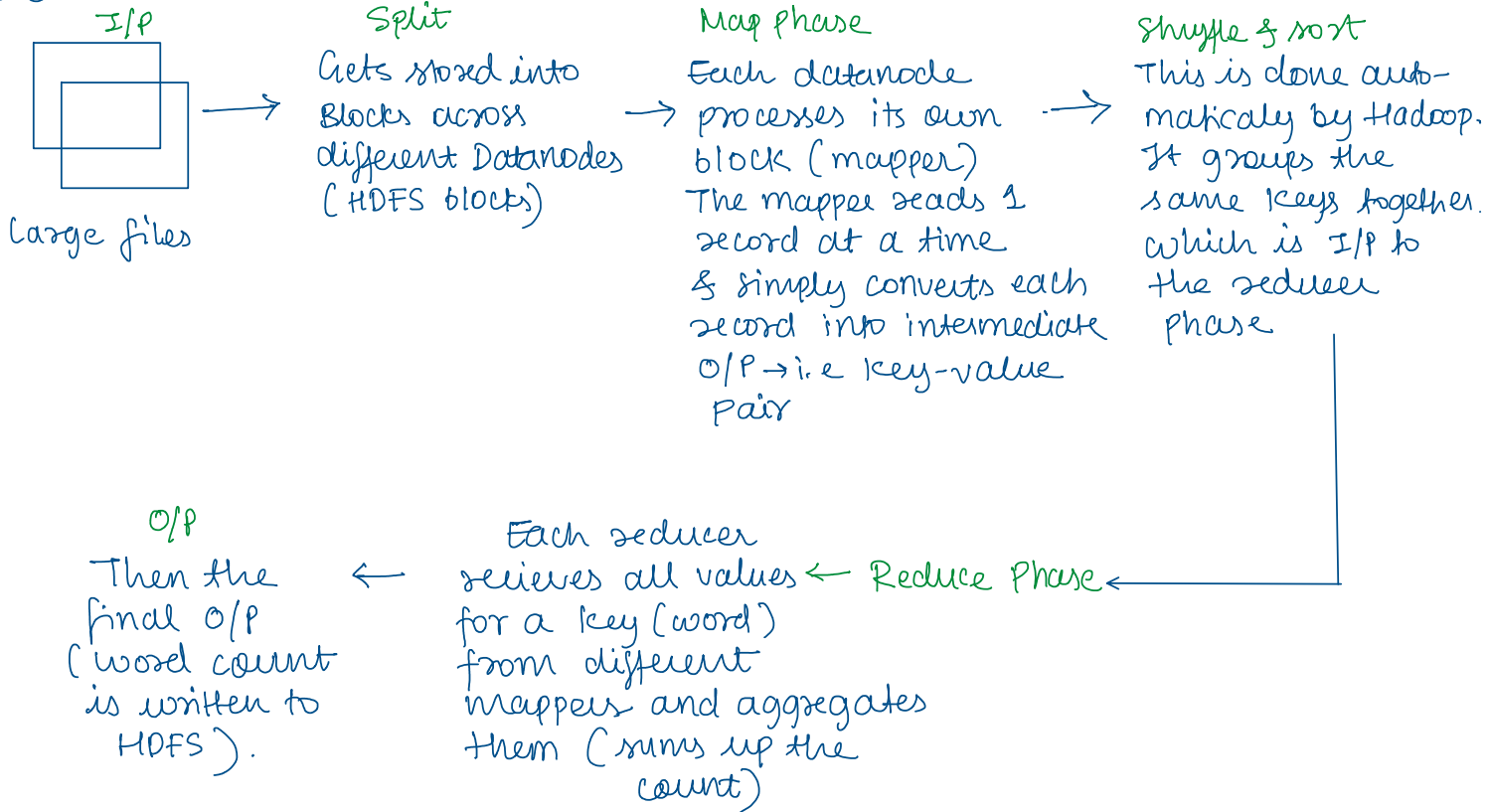


Map Reduce.

Map reduce → is a distributed processing framework that processes large datasets by dividing the work into smaller tasks and running them in parallel across multiple machines.

MapReduce consists of 2 tasks - **Map** and **Reduce**.

Process:-



Example →

Suppose flipkart has 1GB of orders data stored in HDFS :

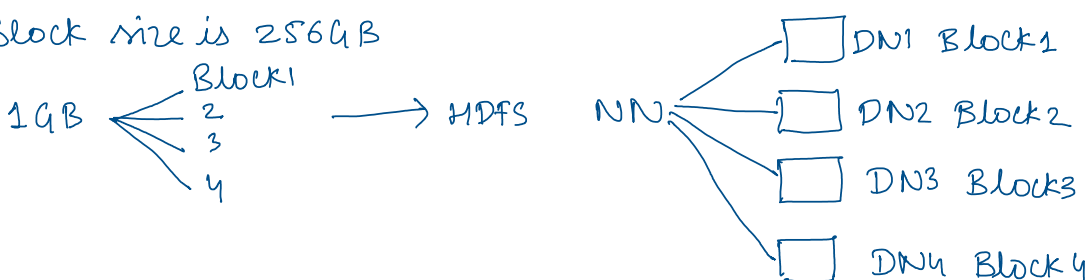
order ID	country	Amount
1.	India	100
2.	US	250
3.	India	300
4.	UK	120
5	India	450
:	US	180

millions of rows -

Business asks → How many orders came from each country?

It's similar to SQL → $\left\{ \begin{array}{l} \text{select country, count(*)} \\ \text{From orders} \\ \text{Group by country.} \end{array} \right\}$

HDFS block size is 256GB

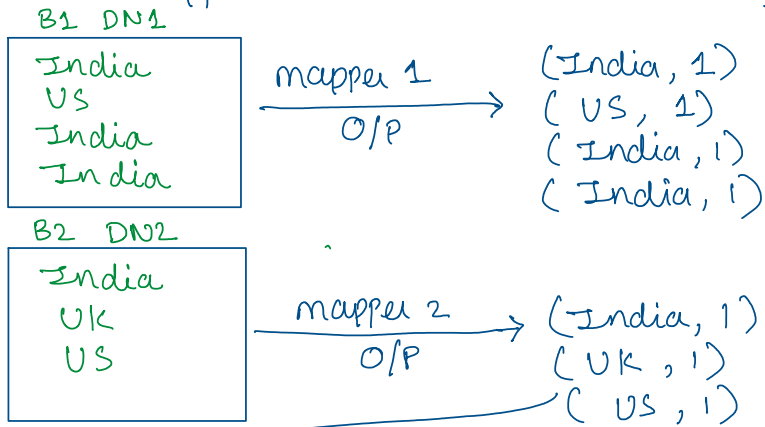


user will submit the Query / mapreduce program to perform the business logic.

Client / user reaches out to NN to get the metadata of the block locations & then each program (mapper) will do the computation

on the block locally (data locality) i.e 4 Blocks stored in 4 Data-nodes, so 4 Mapper will run the computation on 4 blocks placed in different DN's.

Each mapper reads its own block & O/P key-value pair. i.e



Now, hadoop performs shuffle & sort i.e it groups all identical keys together even if they come from different datanodes.

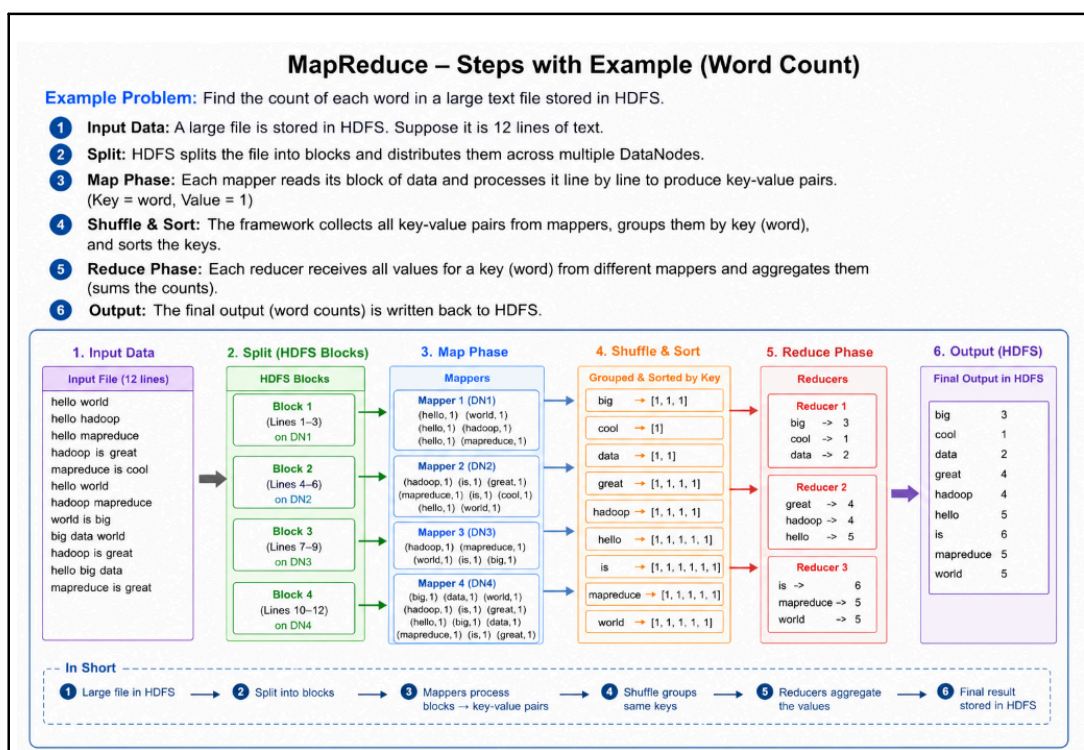
India → [1, 1, 1, 1, 1]
US → [1, 1, 1]
UK → [1, 1]

Now, reducers receives this grouped data

Reducer 1: → India → [1, 1, 1, 1, 1] → O/P India = 6

Reducer 2: → US → [1, 1, 1] → O/P US = 3

All 4 mappers (on each of the DN) works at the same time, so the processing is quite fast → This is parallelism that MapReduce gives.



Q.) How does hadoop decides the number of mapper tasks?

↳ The number of mapper task is determined by the I/P splits not on the number of CPU core

ex: file size: 512 MB

HDFS Block size: 128 MB

The file is divided into 4 blocks, since there are 4 I/P splits, Hadoop creates 4 mapper tasks.

→ In the CPU core will only decide how many mapper task

→ As the CPU core with only 2 available cores, many mapper tasks can be run concurrently
 ex: No. of Blocks = 4
 No. of Mapper task = 4
 Available CPU = 2

execution will happen in waves -

wave 1

Core 1 → mapper 1
 Core 2 → mapper 2

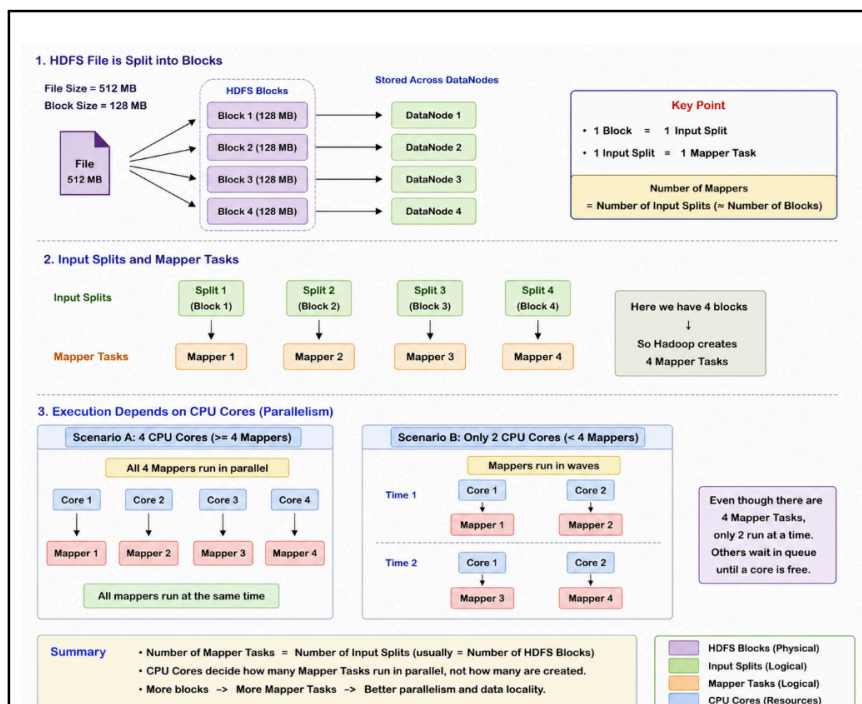
wave 2

Core 1 → mapper 3
 Core 2 → mapper 4

although 2 mappers run in parallel, Hadoop still creates 4 mapper tasks.

when no. of mapper tasks exceeds available CPU core, Hadoop queues remaining mapper tasks. i.e. As soon as the running mapper finishes, the scheduler assigns the next waiting mapper to free CPU.

✗ So, if a cluster has 5 machines each of 4 CPU cores, i.e. total (20 cores) → this does not mean your I/P data will be partitioned into 20 parts only → partition is determined by the block size & CPU cores decides how many tasks can be run concurrently on a cluster. (in this case 20 tasks parallelly). → even if there are 100 Tasks for 100 blocks of data.



1 Mapper task.

i.e. 1 mapper Task processes 1 Block of Data on 1 CPU core

1 GB file (1024)

↳ $1024 / 128$ (default block size)
 ↳ Total blocks = 8
 ↳ 8 mapper tasks.
 ↳ if 4 CPU core, the 4 mapper tasks will run at a time, once it finishes, next 4 will run on the available core. (in batches/batches)

Q) What is the impact of increasing or decreasing the HDFS block size?

Suppose you have 1 GB of I/P file -

Default block size = 128 MB

1 GB → 8 splits → 8 blocks → 8 mapper tasks
 each 128 MB

Case 1: Block size increased to 256 MB

1 GB file → 4 splits → 4 blocks → 4 mapper tasks
 each of 256 MB

Impact → Adv: → Less metadata stored in NN

→ fewer mapper tasks to schedule (low scheduling overhead)

disadv: → Reduced parallelism because of few mappers

→ Some CPU cores may remain idle if the cluster has many available cores

→ A single mapper processes more data, so it may take longer time to process. (longer running mapper tasks)

Let's assume your cluster has 8 CPU cores:

Block size: 256 MB

Blocks: 4

Mappers: 4

Available cores: 8

Core 1-4 running all 4 mapper tasks

Core 5-8 sitting idle

Case 2: Block size is decreased to 64 MB

1 GB file → 16 splits each → 16 Blocks → 16 mapper tasks.

Impact: Adv → Higher parallelism (more mappers)

→ Better CPU utilization if cluster has more cores

→ faster processing & concurrency is increased.

disadv → Increased NN metadata

Higher memory usage on the NN

More mapper tasks increase scheduling overhead

So, Increasing the HDFS block size reduces the no. of blocks & mapper tasks lowering the scheduling overhead but also reducing parallelism.

Decreasing the block size increases the no. of blocks, improving the parallelism but increasing NN metadata & memory usage & task scheduling overhead.

Thus, optimal block size is a balance between efficient parallel processing & minimal overhead.

Advantages of MapReduce -

1) **Parallel Processing** → As mapreduce divides large files into multiple blocks, & each block is processed by a separate mapper - significantly reducing the processing time

Data locality -

2) Instead of moving huge amount of data on 1 server over the network, hadoop runs the mapper on the individual blocks of data stored in distributed way (across datanodes)

↳ i.e. instead of moving data to the processing units we are moving processing unit to the data in mapreduce framework. In traditional system, we used to bring data to PU's & process it but as the data grew & became huge, bringing this much data to PUs has these issues

→ moving huge data to processing is costly & deteriorates N/w performance

→ processing takes a lot of time & it is processed by a single machine (which is a bottleneck)

Now, mapreduce allows to bring the PUs to the data, its adv are -

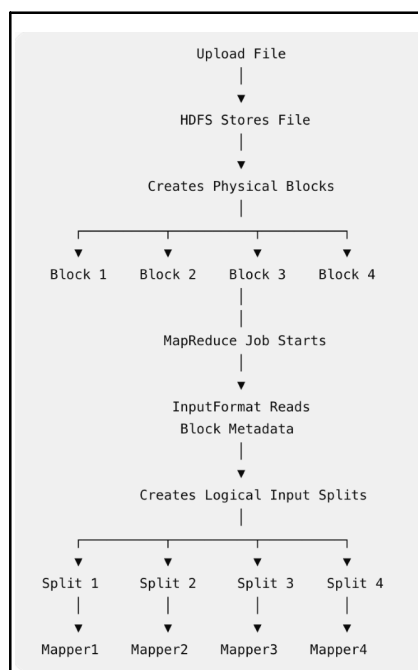
→ cost effective

→ processing time is reduced as all the nodes are working

Mapreduce → when a mapreduce Job starts, it asks "How many mapper tasks should I create"?

↳ & creates I/P splits (it's a pointer to the physical data that 1 mapper has to process; metadata).

→ I/P split says → Mapper 1, read this portion of the file
 ↳ (They simply reference existing HDFS blocks)



HDFS Block	Input Split
Purpose: Store data	Purpose: Process data
Physical part of a file	Logical view of a file
Created when the file is uploaded to HDFS	Created when a MapReduce job starts
Managed by HDFS	Managed by <code>InputFormat</code>
Stored on DataNodes	Not stored anywhere (metadata only)
Exists even if no MapReduce job runs	Exists only during job execution
Used for storage, replication, and fault tolerance	Used to assign work to Mapper tasks
Usually 128 MB (configurable)	Often matches block size, but can differ depending on the <code>InputFormat</code> and file type

↳ so 1 Input split creates 1 Mapper task!

4) **RecordReader** → A recordreader converts the raw texts of I/P splits into key value pairs that mapper can process.

ex:- Raw Text → India, 100

Record reader converts it to

key = 0

value = "India, 100"

(Byte offset)

(Entire line)

Notice: Key is the position (offset) of line in the file.

5) **Mapper** → It processes each I/P record (from RecordReader) and generates new key value pair (and this key-value) is completely different from the input pair.
 O/P of the mapper is known as intermediate O/P which is written to local disk.
 The intermediate O/P is not written into HDFS as this is the temporary O/P data and writing on HDFS will create unnecessary copies.

Suppose the mapper code is →

read line → split by comma → Emit country & amt

I/P: key = 0

value = "India, 100"

mapper O/P :

India → 100

so after mapper

split 1 : India → 100
 USA → 50

split 2 : India → 200
 UK → 80

split 3 : USA → 120
India → 50

These are called intermediate key-value pairs.

6) **Combiner** → It is also known as mini-reducer, it performs the local aggregation on the mapper outputs which help to minimize the data transfer betw mapper & reducer.

ex: Suppose Mapper 3 produced

UK → 120
India → 50
India → 40

→ combiner change to → UK → 120
India → 90

This means fewer records travel across the network, NW traffic decrease & Job becomes fast.

combines values within the same mapper O/P.

7) **Partitioner** → In Hadoop MapReduce, Partitioner comes into the picture if we are working on more than 1 reducer (for 1 reducer partitioner is not used).

↳ A partitioner decides which reducer should receive each key value pair.

It ensures that all the records with same key go to same reducer.

→ How does partitioner decide which key should go on which reducer?

By default Hadoop uses Hash partitioner.

$$\text{partition} = \text{hash}(\text{key}) \% \text{no. of Reducers.}$$

The hash function ensures that the same key always maps to same reducer.

→ for example:

Hash function decides → Reducer 1 Reducer 2
 India USA
 UK

Now every mapper follows this rule & applies to all the O/P.

After partitioning -

Reducer 1 receives

India → 100
India → 200
India → 90

Reducer 2 receives

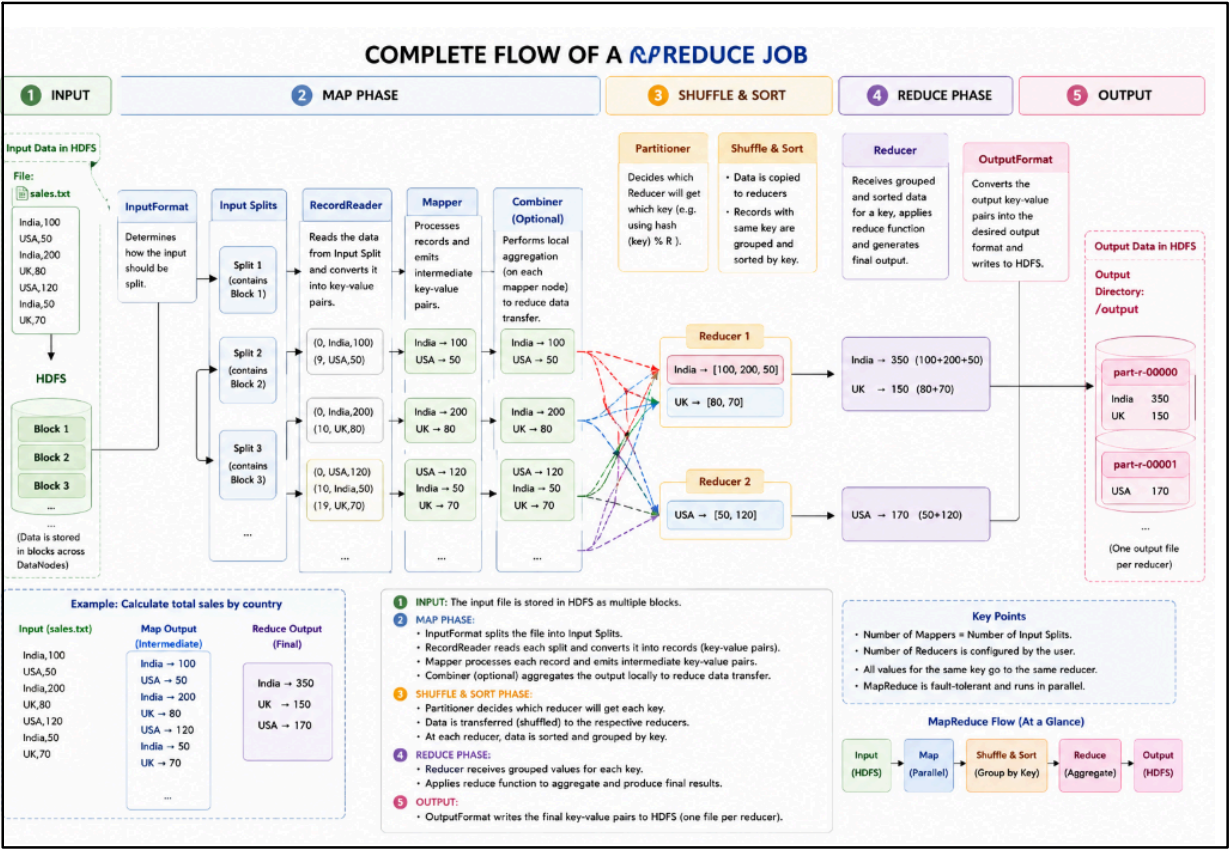
USA → 50
USA → 120

UK → 80

UK → 70

8) **Shuffle & sort** → It collects the data from all mappers, moves it across the network, groups identical keys together & sorts the keys.

9) **Reducer** → It takes the set of intermediate key-value pairs as I/P and then runs a reducer function on each of them to generate O/P.
O/P of the reducer is the final O/P which is stored on HDFS.



YARN

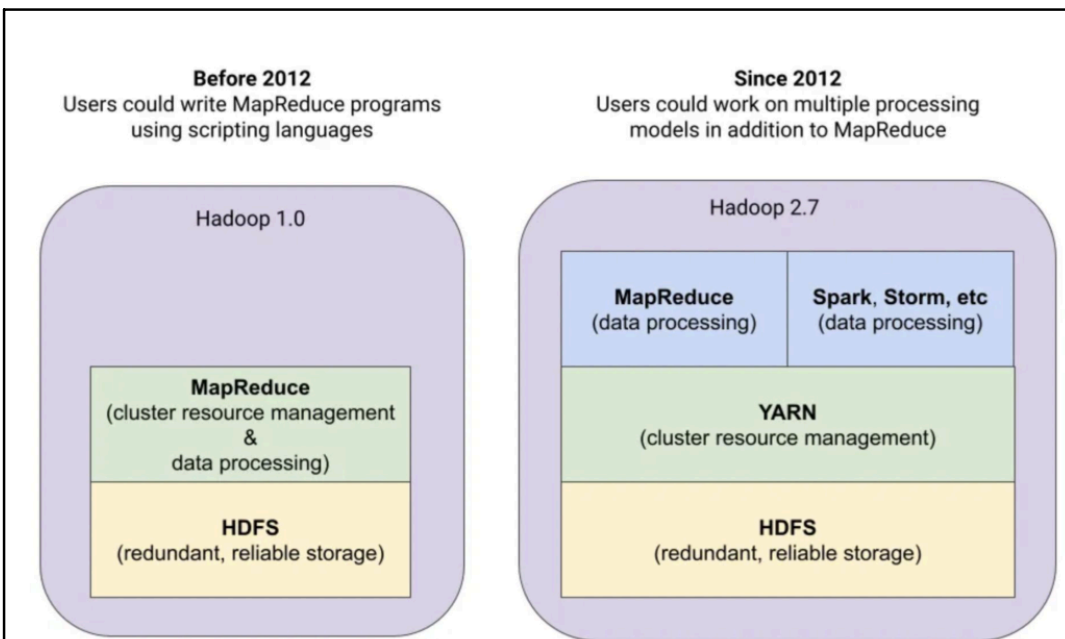
Who will manage the resources (CPU memory storage) on our cluster, who will take care of the Job progress, who will schedule all the incoming Jobs on the clusters?

Before 2012
in Hadoop 1.0
MapReduce use to do the
processing + cluster
resource mgmt both.

after 2012
in Hadoop 2.x

YARN did the cluster mgmt
& Map Reduce / spark / Storm etc
was responsible for data
processing.

it will do the
parallel processing
part only.



Yarn → Yet another resource negotiator is the resource mgmt & Job scheduling layer in Hadoop.

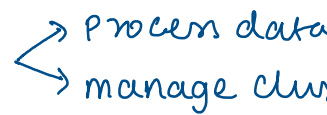
→ It allows multiple data processing engines (like spark, mapreduce etc) to run on the same hadoop cluster by efficiently allocating cluster resources (CPU, memory etc) to applications.

main Job —

- 1) Allocate cluster resources
- 2) Schedule Jobs

- 3) monitor running jobs
4) manage execution of applications like MR, spark, hive etc.

Why was YARN introduced?

- In Hadoop 1.x MR has 2 responsibilities 
- It had to process the data, allocate CPU & memory, schedule jobs everything was handled by MR
- Problem: Suppose you want to run Spark Job on the data stored in HDFS, who will allocate the resources as MR only knew how to run MR jobs
- No spark / other applications couldn't use the hadoop clusters efficiently.
- To solve this, in Hadoop 2.x → it separated the responsibilities:
- HDFS → stores Data
YARN → Manage resources
MR → process data (or any other framework)
- Now any framework can use Hadoop & asks YARN for cluster resources!!

Components of YARN -

(1) **Resource Manager** → It is the master daemon of YARN (i.e. its a daemon service on any node / machine in the cluster). It is responsible for managing resources for several applications (i.e. many teams working on the hadoop cluster will try to send the job & without RM, every application job would try to use the cluster resources at the same time causing conflicts).

Resource Manager (RM) continuously keeps information about every worker node in the cluster - i.e. which node has free memory, which has free CPU, which nodes are busy and which nodes are down.

Functions:-

1) Accepts Application:

Client (You / your system / machine) submits a Job (say sales.txt)
The Job first reaches RM
RM registers the application (Job)

2) Schedule Applications:

Suppose App A needs → 16 GB RAM > But only 20 GB available
App B needs → 10 GB RAM

can both run? NO. RM decides -

Application A → Run ; Application B → Wait

This decision is called scheduling.

3) Allocate Resources:

Suppose Application A needs → 4 CPU & 4 GB RAM, RM searches the cluster & finds Worker 2 is available & allocates the resources (only allocates, does not execute the application).

4) Monitor Cluster resources:

Every worker node sends periodic updates (called heartbeats) to the resource manager

ex:- Worker 1:

"I'm alive"

Free RAM: 10 GB

CPU usage: 40%

& stops assigning work to that node!

If the heartbeat stops, RM assumes worker 1 failed.
(Nodemanager sends heartbeat to RM, RM receives it updates its view of cluster).

RM has 2 internal components -

(I) Scheduler :- It's job is just to decide which application will get resources, how much memory? how many CPU cores? & which containers (resources)
(Think: receptionist handing out room keys)

(II) Application Manager :- It handles the lifecycle of applications. Its responsibilities are:

- accepting new applications/jobs from clients.
- starting application master for each application
- Tracking whether application/job is running or finished
- Cleaning up resources when job completes.

(2) NodeManager - If the RM is the brain/manager of the whole cluster, NodeManager is the manager of one worker machine/datanode.

NodeManager is the worker daemon of Yarn that runs on every worker/datanodes. It is responsible for launching containers, monitor resource usage of its own machine & reporting its health to the RM.

Why Nodemanager is needed?

- Imagine a Company has CEO, 100 Branch offices, each branch has a branch manager. The CEO doesn't go to every branch office to assign work instead Branch manager takes care everything locally.

NodeManager does not know the entire cluster, it knows abt its machine only.

Functions:-

(1) Launch Containers -

Resource Manager allocates Container (Memory 4GB, 2 Core CPU) & RM informs Node manager, NM creates the container & inside it, it starts (mapper/reducer/spark job etc)

So,

RM → Allocates Resources ; NM → Starts the Container
(resources of the machine)

(2) Monitor Containers -

Suppose a container is running → NM keeps checking its memory usage, cpu usage & status & continuously monitors them.

(3) Sends Heartbeat -

Every few seconds NM sends heartbeat to RM, like

RM ← NM 1 (I'm alive, CPU free = 3, RAM free = 8GB
Container Running = 2)

This is How RM will know which resources are available & where in the cluster.

* There's 1 NM per worker node.

* NM does not allocate the resources, Only RM allocates cluster resources & NM simply launches & monitors the containers using the resources assigned to it by RM.

(3) ApplicationMaster (AM) - It is a YARN component that manages the execution of single application. Every application/job

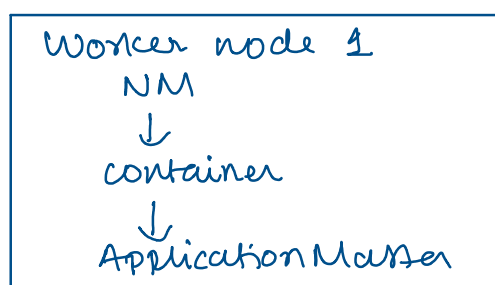
Manages the execution of single application. Every application gets its own AM.

Why do we need AM?

- Suppose 2 users submits Job
Application A (Sales report)
Application B (customer analysis)
- Each application has different no. of I/P files, mappers, reducers execution process
- Who keep tracks of it? AM.
- Hadoop gives each application its own manager i.e AM.

Where does the Application Manager Run?

- It does not run inside the RM.
- RM first allocates one container, inside that container the AM starts



→ The AM is just another process running inside a container.

How many Application Masters are created?

- Suppose 3 users submits 3 applications (Jobs)

User A → Sales Job	} → YARN creates	ApplicationMaster A
User B → customer Job		ApplicationMaster B
User C → Fraud Detection		ApplicationMaster C

Functions:-

(1) **Understands the Job** → Let's say your I/P file has 6 HDFS Blocks. The AM asks namenode - where are the blocks of my I/P file? Namenode replies with metadata of Block location across the worker machines.

Now the AM knows 6 Mapper tasks (RM doesn't calc this)

(2) **Request containers** → Now the AM knows that it needs 6 Mappers to execute Application, so it asks the RM to allocate the required containers (say 6 containers each 2 CPU & 4GB RAM), the RM checks available resources & replies (Approved; Worker 1, 2, 3 each 2 containers)

(3) **Coordinate Task execution** → After the containers are allocated the AM tells NM to start mappers inside it.

AM does not execute the mappers, nodemanager launches the container (as per told by RM) and mappers run inside it, AM only coordinates.

(4) **Monitor progress** → AM monitors the progress of execution of mappers & it knows exactly how much work has finished. it knows →
Total mappers = 6
Completed = 4
Running = 2.

(5) **Handles failure** → Let's say Worker Node 2 crashes, (i.e its NM stops sending heartbeats to the RM & that's why RM detects failure). AM notices that Job inside container of node 2 has failed & requests another container from the RM, once allocated the mapper is restarted on a healthy node.

(4) **Container** → A container is a set of physical resources (CPU, memory (RAM), disks etc) on a single node. → It is allocated by a RM on a worker node to run application task such as AM, mapper, reducer or spark executor.

Why do we need containers?

Suppose your worker node has → 16 GB RAM
8 Core CPU

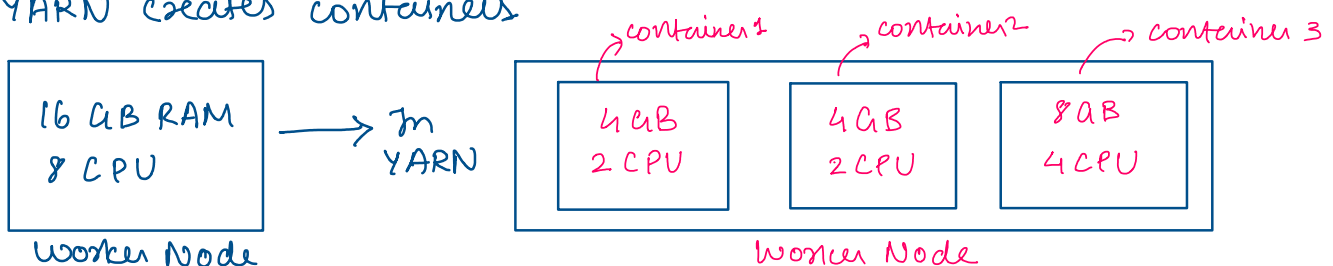
Now 2 applications starts → Application A → needs 8 GB RAM
4 CPU

→ Application B → needs 6 GB RAM
3 CPU

without container, both applications can use as much memory as they want

e.g. App A uses 15 GB RAM } Total → But machine has
App B uses 10 GB RAM } 25 GB RAM only 16 GB, Result
machine crashes X

Instead of allowing applications to use unlimited resources, YARN creates containers

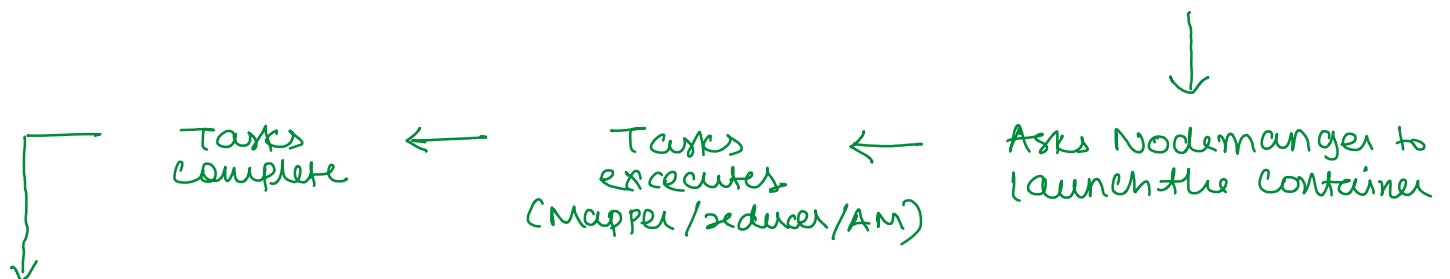


Now every application must run inside a container

Container → Reserved resources for 1 task.

Container lifecycle →

ApplicationMaster → Requests container (resources) → Resource Manager (it allocates container)

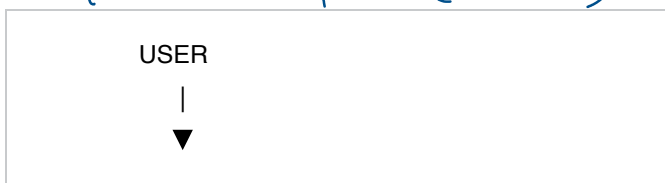


Container released → Resources become available again.

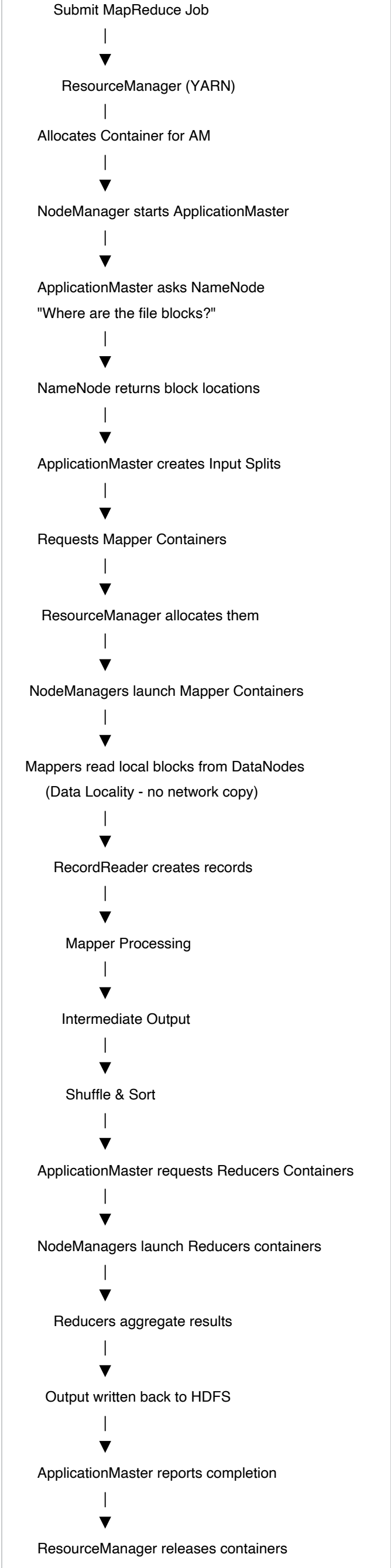
≠ one line →

- **ResourceManager** decides which worker node should get resources.
- **ApplicationMaster** decides what resources its application needs and requests them.
- **NodeManager** creates and monitors the containers on its worker node.
- **Container** is the **resource box (CPU + Memory)** where the actual work (Mapper, Reducer, Spark Executor, or even the ApplicationMaster itself) runs.

Complete Workflow (YARN)



- A client submits a MapReduce application to the **ResourceManager**.



- The ResourceManager allocates a container to launch an **ApplicationMaster**.
- The ApplicationMaster contacts the **NameNode** to get the locations of the HDFS blocks, creates input splits, and requests containers from RM for Mapper tasks.
- The ResourceManager allocates those containers, and the **NodeManagers** launch them on worker nodes— preferably where the corresponding HDFS blocks already exist (**data locality**).
- Each Mapper reads its local block from the **DataNode**, processes the records, and produces intermediate key-value pairs. After the shuffle and sort phase, the ApplicationMaster requests containers from RM for Reducers.
- The Reducers aggregate the grouped data and write the final output back to **HDFS**.
- Finally, the ApplicationMaster notifies the ResourceManager that the job is complete, and all allocated containers are released.